

Problem 1

In problem 3 I made a typo. It is write in my pseduocode, but I just did the cases in the proof for correctness incorrect. I should be $mid > left$ and $mid \leq left$.

Case 2: If $A[mid] > A[mid]$, the left half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[mid]$$

Since the left half is sorted, the transition point (where the maximum wraps to the minimum) cannot lie in the left half. Thus, we update the search to the right half by setting $mid \leftarrow mid + 1$. We know assume Case 2 did not happen.

Case 3: If $A[mid] \leq A[mid]$, the right half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[right] < A[mid]$$

Since the right half is sorted, the transition point must lie in the left half. Thus, we update the search to the left half by setting $right \leftarrow mid$.

Finally, if the while loop terminates without returning (i.e., when $left = right$), the range contains only one element, which must be the maximum. This is because each iteration eliminates half of the array, ensuring the maximum remains in the reduced range. When one element remains, it is guaranteed to be the maximum.

Problem 2

- (a) This problem is equivalent to the Coin Change Problem. Both problems ask for the number of ways to achieve a target sum using a given set of values, where each value has a limited number of occurrences. In the coin change problem, you are given coins of certain denominations (each coin can be used at most as many times as it appears) and you must count the number of unique collections that add up to a specified amount. Similarly, in the submultiset problem, each element in the multiset can only be used as many times as it appears, and you are tasked with counting the number of submultisets that sum to the target t . The constraints in both problems restrict the number of times an item (or coin) can be used, and the goal is to optimize the count of valid combinations that exactly match the target sum.
- (b) This problem is a reformulation of the 0-1 Knapsack Problem. In the 0-1 knapsack problem, you are given a set of items, each with a weight and a value, and the objective is to select a subset of items that maximizes the total value without exceeding a given weight capacity. In the pipe cutting problem, the pipe of length L serves as the capacity, while each available pipe length (with its associated price) corresponds to an item with a “weight” equal to its length and a “value” equal to its price. The restriction that you can sell each length only once mirrors the 0-1 constraint in the knapsack problem, where each item may be selected at most once. Thus, the problem constraints and the objective, to maximize revenue while staying within the total length, directly correspond to the weight capacity and value maximization in the 0-1 knapsack problem.

Problem 3

- (a) (i) Y 's recurrence is flawed because it fails to consider that the desired subset summing to H may be formed by selecting noncontiguous elements from S . In other words, the recurrence only "remembers" if any prefix $\{s_1, \dots, s_j\}$ has already summed to H , and if not, it simply carries forward the result from $P(i-1)$ without considering the possibility of including or excluding the current element s_i to form a new sum that might equal H . This approach overlooks the combinatorial nature of the subset-sum problem, where you must decide, for each element, whether to include it or not. Consider the set $S = \{1, 3, 2, 4\}$ where the total sum is 10, so $H = 5$. Although there exists a valid subset $\{1, 4\}$ that sums to 5, Y 's recurrence would only check contiguous prefixes and find that $P(1)$ is false (since $1 \neq 5$) and $P(2)$ is false (since $1 + 3 = 4 \neq 5$). Thus, it fails to account for the non-contiguous combination $\{1, 4\}$, leading to the incorrect conclusion that no valid partition exists.
- (ii) Look at homework 3 problem 4 part c. Let $f(i, t)$ be the recurrence from that problem where this is the value for if we can get to a sum of size t with the first i elements. The same recurrence would work here we would just use $t = \text{sum}/2$ where sum is the sum of the total array. Since if we know we can find $\text{sum}/2$ this means we know we have a subsequence that sums to $\text{sum}/2$ so take that, and thus we know that the complement will sum up to $\text{sum} - \text{sum}/2 = \text{sum}/2$. Thus, we can split the array into two parts.
- (b) (i) Y 's recurrence is flawed because it assumes that the optimal subarray product in a given range can always be extended contiguously by simply multiplying an endpoint element with the best product from the subarray that excludes that endpoint. In reality, the recurrence uses $S(i+1, j)$ or $S(i, j-1)$ without guaranteeing that the subarray achieving that product actually touches the corresponding boundary ($i+1$ or $j-1$). This means that when you multiply by $A[i]$ or $A[j]$, you might be "gluing together" non-adjacent elements, producing a product that doesn't correspond to any contiguous subarray in the original array. Additionally, the special case where $S(i+1, j) \geq 1$ or $S(i, j-1) \geq 1$ forces a choice of an endpoint element too early, potentially missing a smaller product that could be formed by a proper contiguous combination.
- (ii) Define $P(i)$ to be the minimum product of any contiguous subarray ending at index i . Then the recurrence is

$$P(1) = A[1],$$

$$P(i) = \min\{A[i], A[i] \cdot P(i-1)\} \quad \text{for } i = 2, 3, \dots, n.$$

The justification is as follows: when considering a contiguous subarray ending at $A[i]$, there are only two possibilities. Either the optimal subarray ending at i consists solely of $A[i]$ (i.e., we start fresh at index i), or it is obtained by extending the optimal contiguous subarray ending at $i-1$ by multiplying it by $A[i]$. Since all entries are positive, the product operation is monotonic, and so if $P(i-1) < 1$ it is beneficial to extend the subarray (as $A[i] \cdot P(i-1) < A[i]$); conversely, if $P(i-1) \geq 1$ then starting a new subarray at $A[i]$ is better.

Problem 4

- (a) Lambda can be guaranteed to collect 12 ducks. His optimal strategy is to take the rightmost pile (3 ducks) on his first move. This forces Waquackquack into a position where, regardless of whether he takes the 4 or the 1 duck pile next, Lambda will be able to secure the 9-duck pile on his subsequent turn. Thus, by starting with 3 and then capturing 9, Lambda ensures a total of 12 ducks.
- (b) Let $M(i, j)$ denote the maximum net advantage (the difference between the current player's total ducks and the opponent's total ducks) that the current player can achieve when playing optimally on the subarray of piles from index i to j . The base case occurs when there is only one pile available: for all i , $M(i, i) = D[i]$, since the current player simply takes that pile. For the recursive case, when $i < j$, the player has two options: take the leftmost pile $D[i]$ or the rightmost pile $D[j]$. If the player takes $D[i]$, the opponent will then play optimally on the subarray $[i + 1, j]$, resulting in a net advantage of $D[i] - M(i + 1, j)$; similarly, if the player takes $D[j]$, the net advantage will be $D[j] - M(i, j - 1)$. The reason why we subtract is to simulate it being the other player's turn. It becomes $D[i] - \text{Player 2's Ducks} + \text{Player 1's Ducks} = \text{Player 1's Ducks} - \text{Player 2's Ducks}$. Thus, the recurrence relation is

$$M(i, j) = \max\{D[i] - M(i + 1, j), D[j] - M(i, j - 1)\}.$$

Since there are $O(n^2)$ subproblems and each is solved in constant time, this yields an $O(n^2)$ dynamic-programming solution.

Algorithm 1 Max Ducks

(c) **Require:** An array $D[1, \dots, n]$ where $D[i]$ is the number of ducks at spot i .

Ensure: Finds the max element number of ducks Lambda can get.

```

1: function MAXDUCKS( $D$ )
2:    $M[1, \dots, n][1, \dots, n]$ 
3:   if  $|D| = 1$  then
4:     return  $D[1]$ 
5:   for  $i \in \{1, \dots, n\}$  do
6:      $M[i][i] = D[i]$ 
7:   for  $L \in \{2, \dots, n\}$  do
8:     for  $i \in \{1, \dots, n - L + 1\}$  do
9:        $j \leftarrow i + L - 1$ 
10:       $M[i][j] = \max\{D[i] - M[i + 1][j], D[j] - M[i][j - 1]\}$ 
11:    $sum \leftarrow 0$ 
12:   for  $i \in \{1, \dots, n\}$  do
13:      $sum \leftarrow sum + D[i]$ 
14:   return  $(sum + M[1][n]) / 2$ 

```

- (Proof of Correctness)

We get rid of the base cases in 3,6 these or for when the game is left with one pile. Now onto the for loops. Using the correct recurrence relation from (b) all we have to check is that we do the for loops in the right order. The loop order is correct because it processes the dynamic programming subproblems in an order that respects their dependencies. Initially, the algorithm establishes the base cases by setting $M[i][i] = D[i]$ for all indices i , which represents subarrays of length 1. Then, by iterating over subarray lengths L from 2 up to n , it ensures that when computing $M[i][j]$ for any subarray, the smaller subproblems $M[i + 1][j]$ and $M[i][j - 1]$, corresponding to subarrays of length $L - 1$, have already been computed. This methodical approach guarantees that all necessary values are available when needed for the recurrence $M[i][j] = \max\{D[i] - M[i + 1][j], D[j] - M[i][j - 1]\}$, thereby ensuring the correctness of the computation. Also, for the final return the reason this is correct is for the following reason. First let D_L be Lambda's ducks, and D_W be Waquackquack' ducks. Since:

$$M[1][n] = D_L - D_W, \quad sum = D_L + D_W$$

We get that

$$M[1][n] = 2D_L - sum$$

$$D_L = (M[1][n] + sum) / 2$$

Thus this algorithm is correct.

- (Runtime Analysis)

The algorithm uses a dynamic programming table M of size $n \times n$, and the main computation involves two nested loops. The outer loop runs for $n - 1$ iterations (subarray lengths from 2 to n), and for each outer loop iteration, the inner loop performs at most n iterations, resulting in $O(n^2)$ iterations overall. Additionally, all operations inside the loops take constant time, and the final summing loop adds an $O(n)$ component, so the entire algorithm runs in $O(n^2)$ time.

Problem 5

- (a)
- $(v \rightarrow a)$: $\min(w(v,a), w(v,b) + d(b,a), w(v,c) + d(c,a)) = \min(1, 6, 5) = 1$
 - $(v \rightarrow b)$: $\min(w(v,b), w(v,a) + d(a,b), w(v,c) + d(c,b)) = \min(4, 4, 3) = 3$
 - $(v \rightarrow c)$: $\min(w(v,c), w(v,a) + d(a,c), w(v,b) + d(b,c)) = \min(2, 8, 10) = 2$
 - $(v \rightarrow d)$: $\min(w(v,a) + d(a,d), w(v,b) + d(b,d), w(v,c) + d(c,d)) = \min(6, 8, 7) = 6$

(b)

$$D_{\text{new}}(v_{\text{new}}, u) = \min_{x \in V} \{w(v_{\text{new}}, x) + D(x, u)\}$$

$$D_{\text{new}}(u, v_{\text{new}}) = \min_{x \in V} \{D(u, x) + w(x, v_{\text{new}})\}$$

The correctness of the expressions comes from the observation that any path from v_{new} to a vertex u must start by taking one of the new edges from v_{new} to some vertex x in V and then following the precomputed shortest path from x to u . Similarly, any path from u to v_{new} must end with one of the new edges from some vertex x in V to v_{new} , after traversing the shortest path from u to x . Thus, by minimizing over all possible choices of x in V , the expressions accurately capture the best way to reach or leave v_{new} . Regarding the time complexity, for each vertex $u \in V$ we need to evaluate the sum $w(v_{\text{new}}, x) + D(x, u)$ (or $D(u, x) + w(x, v_{\text{new}})$) for every vertex x in V . Since there are n vertices in V , this computation takes $O(n)$ time for a single u . Repeating this process for all n vertices results in a total running time of $O(n^2)$.

(c)

$$D_{\text{new}}(u, v) = \min(D(u, v), D_{\text{new}}(u, v_{\text{new}}) + D_{\text{new}}(v_{\text{new}}, v))$$

Any u -to- v path either avoids v_{new} (with cost $D(u, v)$) or goes through v_{new} . If it goes through v_{new} , then by optimality we may assume the path is split into the best way to get from u to v_{new} and then from v_{new} to v . Hence, the minimum of these two cases gives the correct answer.

Since for each pair (u, v) the expression requires only a constant number of operations (one addition and one comparison), evaluating it takes $O(1)$ time per pair. As there are $O(n^2)$ pairs in $V \times V$, the total time required to compute $D_{\text{new}}(u, v)$ for all $u, v \in V$ is $O(n^2)$. This, together with the $O(n^2)$ time already used to compute the distances involving v_{new} , yields an overall $O(n^2)$ time for updating the entire distance table of G_{new} .

Problem 6

- (a) A brute-force approach to this problem involves recursively exploring every possible path from the starting vertex (vertex 1) to the destination (vertex n). For each path, you would accumulate the total profit by summing the bids at each visited vertex. This means you start at vertex 1 and, at each vertex, try every outgoing edge to continue the path, keeping track of the profit as you go. Once you reach vertex n , you compare the accumulated profit with the best one found so far, updating it if the current path's profit is higher. By exhaustively enumerating all such paths and recording the maximum profit, you eventually determine the best route for Vikram, even though this approach can take exponential time relative to the number of vertices.
- (b) Let's define a function $P(i)$ that represents the maximum total profit achievable by starting at vertex 1 and reaching vertex i .

Base Case: Since vertex 1 is the starting point, the best profit we can have at vertex 1 is simply the price at vertex 1. Thus,

$$P(1) = p_1.$$

Recurrence Relation: Since the only way to reach i is via one of its predecessors, the best overall profit at i is achieved by taking the best profit from those predecessors and then adding the profit p_i collected at i . Formally, the recurrence is given by:

$$P(i) = p_i + \max_{j \in \text{pred}(i)} P(j),$$

where $\text{pred}(i)$ is the set of all vertices j that have a directed edge (j, i) leading to i .

The recurrence captures the idea that the best way to arrive at vertex i is by choosing the predecessor j that itself has the maximum profit $P(j)$. Since the graph is a directed acyclic graph (DAG) and the vertices are given in topological order, all the $P(j)$ values for $j < i$ are computed before computing $P(i)$. Adding p_i accounts for the profit collected at vertex i itself, ensuring that the profit is accumulated along the entire route.

Algorithm 2 Max Profit

- (c) **Require:** A graph $G = (V, E)$ where each $j \in V$ also has a list of all its incoming edges called $pred(j) \subseteq E$, and an associated $p_j \geq 0$

Ensure: Finds the max profit Vikram can earn.

```

1: function MAXPROFIT( $D$ )
2:    $P[1, \dots, n]$ 
3:   if  $|V| = 1$  then
4:     return  $p_1$ 
5:    $P[1] \leftarrow p_1$ 
6:   for  $i \in \{2, \dots, n\}$  do
7:      $\max_p \leftarrow 0$ 
8:     for  $j \in pred(i)$  do
9:        $\max_p \leftarrow \max\{\max_p, P[j]\}$ 
10:     $P[i] \leftarrow p_i + \max_p$ 
11:  return  $P[n]$ 

```

- (Proof of Correctness)

We prove by induction on the vertex index i (which is in topological order) that the array P stores the maximum profit attainable from vertex 1 to vertex i .

Base Case: For $i = 1$, the algorithm sets $P[1] = p_1$, which is clearly the maximum profit obtainable at the starting vertex.

Inductive Step: Assume that for all vertices $j < i$, the value $P[j]$ correctly represents the maximum profit from vertex 1 to j . Since the graph is a DAG with vertices given in topological order, every path from vertex 1 to vertex i must come through some predecessor $j \in pred(i)$. The best profit that can be accumulated upon reaching i is by selecting the predecessor j that yields the highest profit $P[j]$ and then adding the profit p_i earned at vertex i . The algorithm computes:

$$P[i] = p_i + \max_{j \in pred(i)} \{P[j]\}.$$

By the induction hypothesis, $P[j]$ is correct for each j , and hence $P[i]$ is correctly computed. By induction, the algorithm correctly computes $P[n]$ as the maximum profit obtainable from vertex 1 to vertex n . Also, the reason why we will always have computed $P[j]$ is for all $j \in pred(i)$, $j < i$. Thus, we would have calculated $P[j]$ already so we can use it in $P[i]$.

- (Runtime Analysis)

Outer Loop:

The algorithm processes each vertex from 2 to n in an outer loop, which runs $O(n)$ times.

Inner Loop:

For each vertex i , the inner loop iterates over the set $pred(i)$ of incoming edges. Let $d_i = |pred(i)|$ be the number of incoming edges to vertex i . The time spent for vertex i is therefore:

$$T(i) = c + c' \cdot d_i,$$

where c and c' are constants representing the work done per vertex and per edge, respectively.

Total Work:

The total work done by the algorithm is:

$$T = \sum_{i=1}^n T(i) = \sum_{i=1}^n (c + c' \cdot d_i).$$

This separates into:

$$T = \sum_{i=1}^n c + c' \sum_{i=1}^n d_i = c \cdot n + c' \left(\sum_{i=1}^n d_i \right).$$

Notice that $\sum_{i=1}^n d_i$ is simply the total number of incoming edges over all vertices. In a *DAG*, each edge contributes exactly once to this sum. Therefore,

$$\sum_{i=1}^n d_i = m.$$

So the total running time is:

$$T = O(n) + O(m) = O(n + m).$$

- (d) To find the order all you have to do is look at your array P . Then start at $P(n)$ and go backwards till you find an element s.t. $P(j) = P(n) - p_n$. Then repeat this till you get to 1. This would be your path.

Problem 7

- (a) Let $C[i][j]$ be the min cost("length") of combining string 1 from index 1 to i and string 2 from index 1 to j with preserved order.

Base Cases:

$C[i][0] = i$ since the length has to be just all string 1.

$C[0][j] = j$ same reason as last part.

Recurrence Relation:

$$C[i][j] = \begin{cases} 1 + C[i-1][j-1] & \text{if string 1 at spot } i \text{ is equal to string 2 at spot } j, \\ 1 + \min\{C[i-1][j], C[i][j-1]\} & \text{otherwise.} \end{cases}$$

The reason why this works is that when the characters are the same we just have to add one to the length and look at the min cost to combine both strings minus 1 index. For when they are not the same we have to add one to the length because we have to add one of those characters then there are two possibilities either we added the character from string 1 or from string 2, so we just look at the min of those. Also, the reason why the ordering is correct, and we will not encounter errors where the output has the wrong ordering on the strings is we only go backwards we never look at $i+1$ or $j+1$, so there is no way to add a character at spot i before $k < i$.

This will give us $O(mn)$ since we just have to go through all possibilities of i, j in which there are m or i and n for j .

Algorithm 3 Min Cost

(b) **Require:** Two Strings X, Y s.t. $|X| \leq |Y|$ **Ensure:** Finds the min length of combining the two strings, so they are both sub sequences in order in our new string.

```

1: function MINCOST( $X[0, \dots, n-1], Y[0, \dots, m-1]$ )
2:   if  $|X| = 0$  then
3:     return  $|Y|$ 
4:   if  $|Y| = 0$  then
5:     return  $|X|$ 
6:    $prev[0, \dots, n]$ 
7:    $curr[0, \dots, n]$ 
8:   for  $j \in \{0, \dots, n\}$  do
9:      $prev[j] \leftarrow j$ 
10:  for  $i \in \{1, \dots, m\}$  do
11:     $curr[0] \leftarrow i$ 
12:    for  $j \in \{1, \dots, n\}$  do
13:      if  $X[i-1] = Y[j-1]$  then
14:         $curr[j] \leftarrow 1 + prev[j-1]$ 
15:      else
16:         $curr[j] \leftarrow 1 + \min(prev[j], curr[j-1])$ 
17:     $prev \leftarrow curr$ 
18:  return  $P[n]$ 

```

We compute the table in a row-major order(left to right, top to bottom) so that when calculating the value for $C[i][j]$, all the necessary values, $C[i-1][j]$, $C[i-1][j-1]$, and $C[i][j-1]$, have already been computed. This ordering guarantees correctness because every cell's dependencies have been filled in the previous row or earlier in the current row. Since each value $C[i][j]$ depends only on values from the immediately preceding row (and the current row to the left), we only need to maintain two arrays: one for the previous row and one for the current row. This reduces space usage from $O(mn)$ to $O(n)$ (assuming X has length n and is the shorter sequence).

The run time will be the same since we have two for loops that interlace. $O(nm)$.

Since we update the $prev$ to $curr$ thus overwriting the previous row, we lose the detailed path information needed for reconstruction. Thus, if you want to reconstruct the actual sequence, you would need to either maintain the full $O(mn)$ DP table to backtrack.